

Homework 06: The Set That Loses Things

Part one is a bug hunt you cause on purpose. Part two builds a small hierarchy that uses dispatch honestly.

Part One: Break A HashSet

Create `Library.java` in the classic form.

Write a class `BrokenIsbn` holding a single `String code`. Override `equals` so that two `BrokenIsbn` objects with the same code are equal. **Do not override** `hashCode`.

Then, in `main`:

1. Add the same ISBN to a `HashSet` twice
2. Print the set's size
3. Print whether the set `contains` an equal ISBN
4. Print whether the two objects are `equals` to each other

The output will be contradictory. The set will hold two items it agrees are equal, and will fail to find an item it already contains.

Diagnose It

Write a comment explaining **in your own words** why this happens. A correct answer has to mention buckets. Saying "because hashCode is missing" describes the symptom, not the mechanism.

Then write `Isbn`, the fixed version, and show the same four lines behaving.

The Mutation Trap

Write one more class with a **non-final** field used by both `equals` and `hashCode`. Then:

1. Create an instance and put it in a `HashSet`
2. Print `contains(theObject)`
3. Change the field
4. Print `contains(theObject)` again, and the set's size

The object will be lost inside a set that still counts it. Explain why in a comment, and state what that implies about fields used in `equals`.

Part Two: A Mixed Shelf

Write an abstract `LibraryItem` with a title, and three subclasses: `Book`, `Audiobook` and `Magazine`. Each measures its size differently. A book has pages, an audiobook has a duration, a magazine has an issue number.

Requirements:

- `LibraryItem` has one `abstract` method the subclasses must implement
- `LibraryItem` has one ordinary method they inherit
- `LibraryItem` has one `final` method, with a comment saying why it is final
- One subclass calls `super.someMethod()` to extend rather than replace behaviour
- All three subclasses are `final`, with a comment saying why that is the default

Then loop over an array of `LibraryItem` and print all three. One loop, three behaviours.

Acceptance Criteria

- ☐ Runs with `java Library.java`
- ☐ `BrokenIsbn` output shows size `2`, `contains` `false`, and `equals` `true`
- ☐ A comment explains the bucket mechanism in your own words
- ☐ `Isbn` output shows size `1` and `contains` `true`
- ☐ The mutation trap shows `contains` flipping from true to false without the set's size changing
- ☐ `LibraryItem` has an abstract, an ordinary and a `final` method
- ☐ Every `@Override` is annotated
- ☐ The loop is written against `LibraryItem`, never against a subclass

Stretch

Rewrite `Isbn` as a record and delete `equals`, `hashCode` and `toString`. Confirm the behaviour is identical, then write a comment on what you gave up by switching. There is something, and Module 08 will name it.

Hint, if part one refuses to misbehave

Check that you are constructing **new** objects for the second add and for the `contains` call. Reusing the same variable compares an object to itself, which succeeds by identity alone and hides the bug you are trying to produce.